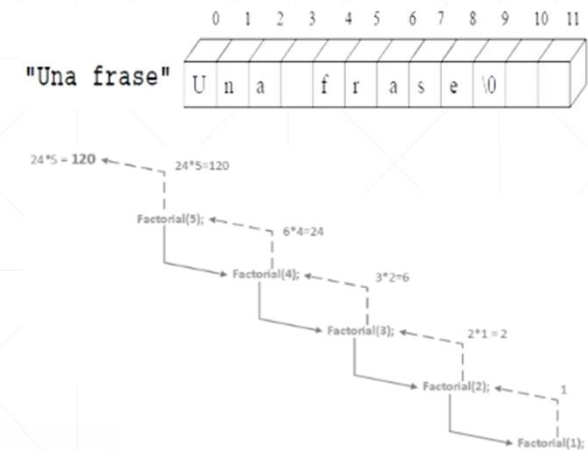




# Manejo de cadenas y recursividad

## Conceptos iniciales

## Que caracteriza a una cadena, para ser cadena?

La barra cero /0

## Como leemos una cadena?

Toda la cadena en sí o carácter por carácter.

## Forma 1: Toda la cadena en sí.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>

int main(){
    char cad[80]="En un lugar de la mancha";
    printf(" %30s\n",cad);
    return 0;
}
```

## Forma 2: Carácter por caracter.

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#include<string.h>

int main(){
    int i;
    char *cad="En un lugar de la Mancha";
    printf("%30s\n",cad);
    printf("Largo de la cadena:%d\n",
        strlen(cad));
    for ( i = 0; i < strlen(cad); i++){
        printf("-%c-",cad[i]);
    }
    return 0;
}
```

## Ejercicio 01: Lectura de una cadena

Ingresa por teclado una cadena de caracteres, mostrarla por pantalla y determinar su longitud mediante una función definida por el usuario llamada LargoCad. Visualizar tanto la cadena ingresada como su longitud.

## Ejercicio 01: Resolución Posible

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>
int LargoCad(char *);

int LargoCad(char *cad){
    int i;
    for (i = 0; cad[i]; i++){
        return i;
    }
}
```



```
int main(){
    unsigned int i;
    int l;
    char cad[81];
    printf("ingrese un Nombre:");
    scanf("%[^\\n]", cad);
    printf("%30s\\n", cad);
    l = LargoCad(cad);
    printf("Largo de la cadena:%d\\n", l);
    for (i = 0; cad[i]; i++){
        printf("-%c-", cad[i]);
    }
    return 0;
}
```

## Ejercicio 02: Memoria dinámica

Definir la cadena cad utilizando memoria dinámica, basándose en el desarrollo realizado en el Ejercicio 1.



## Ejercicio 02: Resolución Posible

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>
int LargoCad(char *);
```

```
int LargoCad(char *cad){
    int i;
    for (i = 0; cad[i]; i++){
        return i;
    }
}
```



```
int main(){
    unsigned int i;
    int l;
    char *cad;
    cad = (char *)malloc(sizeof(char)*81);
    if (cad == NULL){
        printf("No hay memoria disponible");
        exit(1);
    }
    printf("ingrese un Nombre:");
    scanf("%[^\n]", cad);
    printf("%30s\n", cad);
    l = LargoCad(cad);
    printf("Largo de la cadena:%d\n", l);
    for (i = 0; cad[i]; i++){
        printf("-%c-", cad[i]);
    }
    return 0;
}
```

## Ejercicio 03: Modificación de cadenas

A partir del ejercicio anterior, desarrollar una función denominada Mayu que reciba la cadena original y una cadena destino, y devuelva en esta última la cadena original convertida completamente a mayúsculas. Realizar el llamado a la función Mayu() e implementar su definición.

Importante: Declarar siempre los prototipos de las funciones.

## Ejercicio 03: Resolución Posible

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>

void mayu(char[],char[]);

char mayor(char x){
    return((x>='a'&&x<='z')?x-32:x);
}

void mayu(char x[],char y[]){
    int i=0;
    while(x[i]){
        y[i]=mayor(x[i]);
        i++;
    }
    y[i]='\0';
}

int main(){
    unsigned int i;
    int l;
    char cad01[81]; char cad02[81];
    printf("ingrese un Nombre:");
    scanf("%[^\n]", cad01);
    printf("%30s\n", cad01);
    mayu(cad01, cad02);
    printf("%30s\n", cad02);
    return 0;
}
```

## Ejercicio 04: Impresión de cadenas

Basándose en el ejercicio anterior, implementar una función llamada Inverso que reciba la cadena original y su longitud, y muestre la cadena invertida. Realizar el llamado a la función Inverso() e implementar la función correspondiente.

Importante: Declarar siempre los prototipos de las funciones.

## Ejercicio 04: Resolución Posible

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>

int LargoCad(char *cad){
    int i;
    for (i = 0; cad[i]; i++)
    {}
    return i;
}

void inverso(char x[],int y){
    while(y>=0){
        printf("%c",x[y--]);
        //y--;
    }
}
```

Muestra decrementado para no mostrar el último que es el /0  
Y de paso, ya se está guardando. Por eso, comenta el decrementado,  
para no volver a decrementar

```
int main(){
    unsigned int i;
    int l;
    char cad01[81]; char cad02[81];
    printf("ingrese un Nombre:");
    scanf("%[^\n]", cad01);
    printf("%30s\n", cad01);
    l = LargoCad(cad01);
    inverso(cad01, l);
    return 0;
}
```

Así como está, el decrementado de y lo hace luego del procesamiento.  
Sería correcto, en el caso de tener x[--y]. Sin embargo, el profe lo corre bien.  
Habría que decrementar antes del while el primero. Porque ese caracter al ser vacío no lo muestra. Por eso da bien

## Consultas

